DATA & AI PORTFOLIO · PROJECT NOTEBOOK 01

UPI Payments Complaint & SLA Analytics

From raw support tickets to a data-backed root-cause analysis

Author Nikhil Sinha

Repository github.com/Nikhil201716/01-UPI-Payments-Complaint-SLA-Dashboard

Stack Python · pandas · SQLite · SQL · openpyxl · Streamlit

Edition 1.0

How to read this book

This book is written to be read from the beginning by someone who has not seen the project and does not yet know SQL, SLA mathematics, or how a complaints operation works. It is not API documentation. It is closer to a course text built around one real system.

Chapters 1 to 3 establish the problem and the data. **Chapters 4 to 7** are the technical core - relational modelling, SQL, the statistics of rates, and root-cause method - and each teaches the subject before applying it. **Chapter 8 is the runbook**: every terminal command needed to reproduce everything, with expected output. **Chapters 9 to 11** cover the delivery artefacts and the results.

Two conventions are used throughout. Coloured callouts marked **WHY** explain a design decision and the alternative that was rejected; callouts marked **WARNING** mark a trap that is easy to fall into and hard to notice afterwards. Every code listing is extracted from the repository when this book is built, so nothing here can drift out of step with the code it describes.

All data in this project is synthetic. No real customer, transaction, or company data appears anywhere. Chapter 2 documents exactly how it is generated and what was deliberately built into it.

Contents

16. Appendix

CHAPTER 1

The Business Problem

> *Before a single line of code: what a UPI complaints operation actually is, who is accountable for what, and why the difference between a 75% and a 90% SLA number is a regulatory matter rather than a dashboard colour.*

1.1 What UPI is, and why complaint volume is structural

The Unified Payments Interface is India's real-time retail payments rail. A customer initiates a transfer from an app - Google Pay, PhonePe, Paytm - and the money moves between two bank accounts in seconds, mediated by the National Payments Corporation of India (NPCI). At the scale UPI operates, even a very small failure rate produces an enormous absolute number of unhappy customers.

This matters for the shape of the problem. A payments support queue is not driven by product defects that can be fixed once and closed. It is driven by a *rate* applied to a very large denominator: gateway timeouts, bank downtime, user error, and genuine fraud all produce complaints continuously and predictably. The operational question is therefore never 'how do we stop complaints', it is 'given that this volume will arrive, are we resolving it within the time we promised'.

That promise is the Service Level Agreement, and it is the spine of this entire project.

1.2 Service Level Agreements as a contract, not a metric

An SLA is a commitment of the form: *we will resolve at least X% of complaints of type T within H hours*. Three components matter and are frequently confused.

- **The target window (H)** - different complaint types get different windows. A failed transaction awaiting reversal is a 24-hour promise; a fraud dispute requiring investigation is 168 hours. Applying one window to all types is the single most common modelling error in this domain.
- **The compliance threshold (X)** - typically 90%. Note that this is not 100%: the SLA explicitly accepts that a minority of cases will be complex.
- **The measurement population** - which tickets count. Do you include tickets still open? Tickets reopened? Tickets the customer abandoned? Each choice moves the number, and a team that is not explicit about it can 'improve' compliance without changing anything real.

WHY THIS PROJECT DEFINES THE POPULATION EXPLICITLY

The pipeline computes compliance only over *closed* tickets with a valid resolution timestamp. Open tickets have no resolution time and cannot be assessed; silently treating them as compliant would inflate the number, and treating them as breaches would deflate it. Excluding them and stating the exclusion is the only defensible option, and Chapter 6 quantifies how much it matters.

1.3 The regulatory frame: why 75.7% is not merely disappointing

The Reserve Bank of India's Ombudsman Scheme for Digital Transactions (2019) gives customers a formal escalation path when a payment system operator fails to resolve a complaint within a defined period. That changes the character of an SLA breach: it is not only a customer-experience cost, it is the trigger for an external process with reporting obligations attached.

The complaint taxonomy used throughout this project is modelled on that scheme's categories - failed transactions where the account was debited, unauthorised or fraudulent transactions, delayed refunds, and misdirected transfers - rather than being invented. Using the regulator's own vocabulary means the analysis maps onto the categories the business is actually judged against.

> **A WORD ON THE DATA**
>
> Every record in this project is synthetically generated. No real customer, transaction, or company data appears anywhere. The generator is described in full in Chapter 2, including the mechanisms deliberately injected into it, because a synthetic dataset whose construction is hidden cannot be reasoned about.

1.4 The three questions this project answers

Operations leadership needs three things, in order:

1. **Which complaint categories breach SLA the most, and why?** A ranked, quantified list - not an impression.
2. **Is the queue improving or deteriorating?** A trend with enough granularity to separate a genuine shift from normal month-to-month noise.
3. **What specific change would move the number?** A recommendation tied to a mechanism, with the expected size of the effect.

Everything downstream - the schema, the queries, the dashboards, the written analysis - exists to answer those three questions and nothing else. Chapter 12 returns to them and states how completely each was answered, including where the answer is weaker than it looks.

1.5 Who reads what

A common failure of analytics projects is producing one artefact for four audiences. This project produces four, deliberately:

Table 1.1 The four deliverables and their distinct audiences

Artefact	Audience	Question it answers	Format
Excel dashboard	Ops managers, weekly review	Where are we against target this week?	XLSX, offline, emailable
Streamlit app	Analysts, ad-hoc investigation	Why did this category move?	Interactive, filterable
SQL query file	Data team, reuse and audit	How exactly was this computed?	Versioned SQL
Root-cause report	Leadership, decision-making	What should we change?	Written argument with numbers

WHY EXCEL IS A FIRST-CLASS DELIVERABLE

Engineers routinely dismiss Excel. In payments operations it remains the format that a manager can open on a phone, forward to a regulator, and annotate without installing anything. A dashboard that requires a running Python process is not available at 7am on a Sunday when an incident review is happening. Chapter 9 builds the Excel artefact properly, with real conditional formatting rather than a data dump.

CHAPTER 2

Synthetic Data Generation

> *How to build a dataset that behaves like reality without using anyone's real data - and why a seeded generator with documented mechanisms is more useful for learning than a real extract would be.*

2.1 Why synthetic data is the right choice here

There is no lawful, ethical route to a real payments complaint dataset for a portfolio project. Beyond that constraint, synthetic data has a property real data lacks: **the ground truth is known**. When the generator deliberately makes the Fraud & Disputes queue slower, any analysis that fails to detect that fact is demonstrably wrong, and the failure is diagnosable.

This turns the project from 'here are some findings' into 'here are findings that can be checked against what was actually injected'. That distinction runs through every project in this portfolio.

THE TRAP OF SYNTHETIC DATA

A generator that draws every field independently produces a dataset with no structure to find. The analysis then discovers nothing, or worse, discovers noise and reports it confidently. The generator here injects specific, documented dependencies - category drives SLA target, team drives resolution speed, calendar position drives volume - and every one of them is recoverable by the analysis.

2.2 Determinism: the seed, and why it is not enough

The generator fixes a seed so the dataset is identical on every run:

Listing 2.1 - the seed — `scripts/generate_data.py`

```
SEED = 42
```

A fixed seed makes the sequence of random draws reproducible. It does **not** by itself make the output reproducible, and this is a subtlety that cost two later projects in this portfolio a full debugging session each.

WHERE DETERMINISM SILENTLY BREAKS

If code draws from a Python `set`, or uses the builtin `hash()` on strings, the iteration order varies between *processes* even with a fixed seed, because Python randomises string hashing per interpreter run. The data is then perfectly stable within one run and different the next morning.

The defence used across this portfolio is to sort any collection before it feeds a random draw, and to verify reproducibility by comparing SHA-256 checksums computed in a *fresh process* - never by re-running in the same one, which cannot detect the bug.

2.3 The complaint taxonomy and its SLA windows

Each complaint category carries its own target window, mirroring the regulatory framework. The windows are not arbitrary: they encode how much investigation the category genuinely requires.

Table 2.1 Complaint categories and their SLA targets

Category	Target (hours)	Why that window
Failed Transaction - Amount Debited	24	the money exists and must be returned; largely mechanical
Delayed Refund	72	depends on a merchant or bank counterparty
Wrong Beneficiary Credited	120	requires contacting the unintended recipient's bank
Unauthorized Transaction / Fraud	168	requires genuine investigation, possibly law enforcement

Modelling the window as a per-category constant rather than a global one is what makes the breach analysis meaningful. A fraud case resolved in 100 hours is *compliant*; a failed-transaction reversal resolved in 100 hours is a serious breach. A single global threshold would score both identically and produce nonsense.

2.4 Modelling resolution time: why lognormal, not normal

Resolution time is the central quantity in the whole project, so its distribution deserves an argument rather than a default.

The case against a normal distribution

A normal distribution is symmetric and unbounded below. Applied to resolution time it produces negative durations, which are meaningless, and it makes very long resolutions as rare as very short ones, which is the opposite of what queues do.

The case for lognormal

A duration is the product of many multiplicative effects - how quickly the ticket was picked up, multiplied by how many handoffs it took, multiplied by whether a counterparty responded. When independent effects multiply, their logarithm sums, and the Central Limit Theorem applies to that sum. The result is that the duration itself is lognormally distributed:

$$T = e^{\mu + \sigma Z}, \quad Z \sim \mathcal{N}(0, 1)$$

This gives exactly the behaviour a support queue exhibits: strictly positive, a dense mass of quick resolutions, and a long right tail of cases that took far longer than typical.

MEAN AND MEDIAN OF A LOGNORMAL, AND WHY THEY DIFFER

For $T \sim \text{Lognormal}(\mu, \sigma^2)$:

$$\text{median}(T) = e^{\mu}, \qquad \mathbb{E}[T] = e^{\mu + \sigma^2/2}$$

The mean always exceeds the median, and the gap grows with σ . This is not a curiosity - it is why the SQL layer reports both. A team whose mean resolution time is 40 hours and whose median is 12 hours does not have a 40-hour problem; it has a small number of very stuck cases dragging an average that describes almost none of its work.

2.5 Injecting the mechanisms the analysis must find

Three dependencies are built into the generator on purpose. Each one is a finding the analysis is expected to recover.

1. **Team capability differs.** Technical Escalations resolves faster than Tier-1 Support, which resolves faster than Fraud & Disputes. This is realistic - fraud work is genuinely harder - and it makes 'which team is slowest' a question with a correct answer.
2. **Volume follows the salary cycle.** Transaction volume in India spikes around payday, and complaint volume follows it. The generator raises arrival intensity in the 25th-2nd window, so the trend analysis has a real periodic signal to detect rather than noise.
3. **A minority of tickets are never assigned.** These have no owner actively watching the clock and therefore breach more often - a structural effect rather than a performance one.

WHAT THE ANALYSIS RECOVERED

All three were detected. Fraud & Disputes came out at a 40.9% breach rate against 6.0% for Technical Escalations; the salary window showed 53.5 tickets/day against 39.3 outside it, a 36% spike; and the 235 unassigned tickets breached at 29.8%, worse than any staffed team. Chapter 12 gives the full results with the caveats attached.

2.6 Deliberately dirtying the data

Real operational exports are not clean, and a pipeline that has never met a malformed timestamp will meet one in production. The generator therefore writes timestamps in several formats and leaves some fields missing:

Listing 2.2 - deliberate format inconsistency `scripts/generate_data.py`

```
def messy_timestamp(ts):
```

Chapter 3 is entirely about surviving this. The important design point is that the mess is *generated deliberately and documented*, so the cleaning layer can be tested against a known answer rather than tuned until the errors stop appearing.

CHAPTER 3

Data Cleaning and Transformation

> *Parsing timestamps that arrive in four formats, deciding what a missing value means, and computing the derived fields the entire analysis depends on.*

3.1 The parsing problem

The raw extract contains timestamps in multiple formats, because it was assembled from several upstream systems - which is exactly what happens when a support tool, a payments gateway, and a CRM each export their own convention. A single `pd.to_datetime` call will either fail or, worse, silently coerce some values to `NaT` and carry on.

Listing 3.1 - format-by-format parsing with an explicit fallback `scripts/clean_transform.py`

```
def parse_mixed_timestamp(value):
```

WHY EXPLICIT FORMATS BEAT AUTOMATIC INFERENCE

Automatic inference is ambiguous for dates like `03/04/2026`, which is 3 April under one convention and 4 March under another. Inference will pick one per *column* based on what it sees first, so a column that begins with unambiguous dates can flip interpretation partway through a file. Trying known formats in a defined order removes the ambiguity, and anything matching none of them is recorded as unparseable rather than guessed.

3.2 Missingness: three different meanings, three different treatments

A blank cell is not a single concept. This pipeline distinguishes:

Table 3.1 Missingness is semantic, not uniform

Field	What blank means	Treatment
resolved_at	ticket is still open	keep the row, exclude from SLA maths
assigned_team	never routed to anyone	recode as 'Unassigned' - it is a finding
transaction_amount	not captured at intake	leave null, exclude from value sums

A MISSING VALUE THAT TURNED OUT TO BE THE THIRD ROOT CAUSE

The instinctive treatment of a blank `assigned_team` is to drop the row or fill it with the mode. Both would have destroyed the finding: those 235 tickets breach at 29.8%, worse than any actual team, precisely *because* nobody owned them. The missingness was the signal. Recoding it as an explicit category rather than erasing it is what let the analysis see it.

3.3 Derived fields: resolution time and breach flag

Two computed columns carry the entire analysis. Resolution time first:

Listing 3.2 - resolution time in hours — scripts/clean_transform.py

```
def compute_resolution(row):
```

Then the breach flag, which compares that duration against the category-specific target from Table 2.1:

Listing 3.3 - per-category SLA breach — scripts/clean_transform.py

```
def compute_breach(row):
```

THE COMPARISON IS STRICTLY GREATER-THAN

A ticket resolved in exactly 24.0 hours against a 24-hour target is *compliant*, not breached. The boundary case is a genuine decision, not an accident of code: the SLA promises resolution 'within' the window, and the instant the window ends is still within it. Getting this backwards moves the compliance number by however many tickets land exactly on the boundary, which in a system with hour-granularity timestamps is not negligible.

3.4 Why cleaning is a separate stage with its own output

The pipeline writes a cleaned dataset to disk rather than cleaning inside the analysis. Three reasons:

- **Auditability** - the cleaned file can be inspected and diffed. A transformation buried inside an analysis script is invisible to review.
- **Cost** - parsing 15,700 rows with multiple format attempts is not free, and the analysis reruns far more often than the source data changes.
- **Blame isolation** - when a number looks wrong, the first question is whether it entered wrong or was computed wrong. A materialised intermediate answers that in seconds.

CHAPTER 4

A Guided Tour of the Codebase

> *Every script in the repository, read in the order the pipeline runs them, with the design decision behind each one made explicit.*

4.1 The orchestrator

Reading a codebase is easiest in execution order, and the orchestrator defines that order. It is deliberately the shortest file in the project: its only job is sequencing, and any logic that leaks into it becomes logic that cannot be run or tested on its own.

Listing T.1 - the full orchestrator — scripts/run_pipeline.py

```
"""
run_pipeline.py
---------------
One-command orchestrator for the full ETL pipeline. Running this single
script regenerates everything downstream from one source of truth -
synthetic data -> cleaned data -> SQLite database -> Excel dashboard -
so nothing ever has to be manually re-entered across systems.

Usage:
    python scripts/run_pipeline.py
"""

import subprocess
import sys
from pathlib import Path

SCRIPTS_DIR = Path(__file__).resolve().parent
ROOT = SCRIPTS_DIR.parent

STEPS = [
    ("Generating synthetic complaint data", SCRIPTS_DIR / "generate_data.py"),
    ("Cleaning & transforming raw data", SCRIPTS_DIR / "clean_transform.py"),
    ("Building SQLite database", SCRIPTS_DIR / "build_database.py"),
    ("Validating SQL analysis queries", SCRIPTS_DIR / "run_queries_check.py"),
    ("Building Excel dashboard", ROOT / "dashboard" / "build_excel_dashboard.py"),
]

for i, (label, script) in enumerate(STEPS, start=1):
    print(f"\n{'=' * 70}\nSTEP {i}/{len(STEPS)}: {label}\n{'=' * 70}")
    result = subprocess.run([sys.executable, str(script)], cwd=script.parent)
    if result.returncode != 0:
        print(f"\nPipeline FAILED at step {i}: {label}")
        sys.exit(1)

print(f"\n{'=' * 70}\nPipeline completed successfully.")
print("Next: run the interactive dashboard with:")
print("    streamlit run dashboard/streamlit_app.py")
print(f"{'=' * 70}")
```

WHY EACH STAGE IS A SEPARATE PROCESS

Running each stage as its own subprocess rather than importing and calling it means a stage cannot leave global state behind for the next one, a crash is attributable to exactly one stage, and any stage can be re-run alone during development. The cost is process startup time, which at this scale is irrelevant.

4.2 The generator: configuration and taxonomy

The generator is the longest script in the project because it encodes the entire domain model. Its structure is: define the taxonomy and its SLA windows, define the teams and their capability, then draw tickets.

Listing T.2 - configuration and taxonomy scripts/generate_data.py

```
"""
generate_data.py
----------------
Generates a realistic SYNTHETIC dataset of digital-payment (UPI) customer
support complaints, modeled on:
  - The complaint category taxonomy published by the RBI Ombudsman Scheme
    for Digital Transactions, 2019.
  - Approximate Turn-Around-Time (TAT) windows loosely based on RBI's
    "Harmonisation of TAT and customer compensation for failed transactions
    using authorised Payment Systems" circular (RBI/2019-20/67).
  - Realistic support-operations patterns observed in live digital-payments
    customer support work (volume spikes around salary dates, incident days
    causing technical-error surges, fraud/dispute cases taking longest to
    resolve).

No real customer, transaction, or company data is used anywhere in this
project. All records are synthetically generated for portfolio purposes.

Output: 4 CSV files written to ../data/
  - dim_category.csv
  - dim_channel.csv
  - dim_agent.csv
  - fact_complaints.csv
"""

import numpy as np
import pandas as pd
from pathlib import Path
from datetime import datetime, timedelta

# ---------------------------------------------------------------------
# 0. Setup
# ---------------------------------------------------------------------
SEED = 42
rng = np.random.default_rng(SEED)

DATA_DIR = Path(__file__).resolve().parent.parent / "data"
DATA_DIR.mkdir(exist_ok=True)

START_DATE = datetime(2025, 8, 1)
END_DATE = datetime(2026, 7, 30)  # "today"
TOTAL_DAYS = (END_DATE - START_DATE).days + 1

# ---------------------------------------------------------------------
# 1. Dimension: Complaint categories
#    sla_target_hours are illustrative TAT windows, not an exact
#    reproduction of any bank's or regulator's real published table.
# ---------------------------------------------------------------------
categories = pd.DataFrame([
    # category_id, category_name, category_group, sla_target_hours, base_weight
    (1, "Transaction Failed - Amount Debited (Auto-Reversal Pending)", "Failed Transaction", 24,
0.20),
    (2, "Refund Delayed / Not Received",                               "Refunds",            120,
0.16),
    (3, "Unauthorized Transaction / Fraud Dispute",                    "Fraud & Disputes",    168,
0.08),
    (4, "Amount Debited but Not Credited to Beneficiary",              "Failed Transaction", 48,
```

```
0.13),
    (5, "Duplicate Debit for Same Transaction",                    "Failed Transaction", 72,
0.07),
```

The SLA windows and category weights are module-level constants rather than literals scattered through the drawing code. This is the same single-source-of-truth discipline that later projects formalise into a dedicated `domain.py`: a value written in two places will eventually disagree with itself.

4.3 The generator: drawing the population

Listing T.3 - drawing the ticket population scripts/generate_data.py

```
    (5, "Duplicate Debit for Same Transaction",                          "Failed Transaction", 72,
0.07),
    (6, "KYC / Account Verification Issue",                              "Account & KYC",       96,
0.06),
    (7, "Merchant / QR Payment Failure",                                 "Merchant Payments",   48,
0.11),
    (8, "App Technical Error (Login / OTP Blocking Payment)",            "Technical",           24,
0.10),
    (9, "Wrong Beneficiary Credited (Misdirected Transfer)",             "Fraud & Disputes",    120,
0.04),
    (10, "Cashback / Reward Not Credited",                               "Rewards",             168,
0.05),
], columns=["category_id", "category_name", "category_group", "sla_target_hours", "base_weight"])

# Root causes each category can be tagged with once resolved
ROOT_CAUSES = {
    1: ["Bank-side auto-reversal delay", "Payment gateway timeout", "NPCI switch downtime", "Manual
reversal required"],
    2: ["Refund batch job delay", "Bank processing delay", "Incorrect refund account on file",
"Merchant refund not initiated"],
    3: ["Confirmed phishing/OTP-sharing fraud", "SIM-swap fraud", "Card-not-present fraud", "False
positive - user forgot transaction"],
    4: ["Beneficiary bank server issue", "IFSC/account mismatch", "NPCI settlement delay", "Beneficiary
account frozen"],
    5: ["Network retry causing duplicate debit", "User double-tap on payment button", "Merchant
terminal glitch"],
    6: ["Document mismatch", "PAN-Aadhaar link pending", "Video KYC slot unavailable", "Manual review
backlog"],
    7: ["Merchant QR misconfigured", "Merchant bank account inactive", "POS/QR gateway timeout"],
    8: ["App version bug post-release", "OTP delivery delay from telecom", "Server outage", "Login
token expiry bug"],
    9: ["User entered wrong UPI ID", "Similar beneficiary name confusion", "Contact sync error in
app"],
    10: ["Rewards batch processing delay", "Promotion terms not met", "Cashback engine bug"],
}

# ---------------------------------------------------------------------
# 2. Dimension: Channels
# ---------------------------------------------------------------------
channels = pd.DataFrame([
    (1, "UPI App-to-App Transfer"),
    (2, "Merchant QR Payment"),
    (3, "Bill / Recharge Payment"),
    (4, "Bank-Linked Bank Transfer"),
    (5, "Peer-to-Peer (Contact) Transfer"),
], columns=["channel_id", "channel_name"])

# ---------------------------------------------------------------------
# 3. Dimension: Agents (3 teams)
# ---------------------------------------------------------------------
FIRST_NAMES = ["Aarav","Vivaan","Aditya","Ananya","Diya","Ishaan","Kabir","Meera",
               "Neha","Om","Pooja","Rohan","Sana","Tara","Uday","Varun","Yash",
               "Zara","Kavya","Arjun","Priya","Rahul","Sneha","Karan","Isha"]
TEAMS = ["Tier-1 Support", "Fraud & Disputes", "Technical Escalations"]

n_agents = 24
agents = pd.DataFrame({
```

```
    "agent_id": range(1, n_agents + 1),
    "agent_name": [f"{FIRST_NAMES[i % len(FIRST_NAMES)]} {chr(65 + (i * 7) % 26)}." for i in
range(n_agents)],
    "team": [TEAMS[i % 3] for i in range(n_agents)],
})

# ---------------------------------------------------------------------
# 4. Simulate "incident days" - real app outages / bad releases
#    These days cause a big spike in Technical + Failed-Transaction
#    complaints, which we deliberately use later for root-cause analysis.
# ---------------------------------------------------------------------
incident_days = [
    datetime(2025, 10, 14),   # simulated bad app release
    datetime(2026, 1, 5),     # simulated NPCI switch outage (new year traffic)
    datetime(2026, 5, 22),    # simulated server outage
]

# ---------------------------------------------------------------------
# 5. Decide daily ticket volume per day (seasonality + growth + incidents)
# ---------------------------------------------------------------------
dates = [START_DATE + timedelta(days=i) for i in range(TOTAL_DAYS)]
daily_volumes = []
base_volume = 32
```

READING GENERATOR CODE CRITICALLY

When reviewing any synthetic generator, the question to ask at every line is: *does this create a dependency the analysis is supposed to find, or does it create noise?* Both are legitimate, but confusing them produces a dataset where the analysis appears to work and is actually finding nothing.

4.4 The cleaner

The cleaning script has three responsibilities and keeps them separate: parse, decide missingness, derive. Each is a function, which means each is independently testable and independently wrong when it is wrong.

Listing T.4 - imports and the accepted format table scripts/clean_transform.py

```
"""
clean_transform.py
-------------------
Takes the deliberately messy raw export (fact_complaints_raw.csv, which has
mixed timestamp formats, duplicate rows, missing agent assignments, and
stray whitespace/casing issues - exactly like a real ticketing-system
export) and produces a single clean, analysis-ready CSV.

This is the same "standardize mixed-format timestamps to ISO 8601" skill
used on the Uber Supply-Demand Gap Analysis project, applied here to a
second, independent messy dataset.

Input:  ../data/fact_complaints_raw.csv
Output: ../data/fact_complaints_clean.csv
"""

import pandas as pd
import numpy as np
from pathlib import Path

DATA_DIR = Path(__file__).resolve().parent.parent / "data"

raw = pd.read_csv(DATA_DIR / "fact_complaints_raw.csv")
print(f"Raw rows loaded: {len(raw):,}")

# ---------------------------------------------------------------------
# 1. Standardize mixed-format timestamps to ISO 8601 (YYYY-MM-DD HH:MM:SS)
#    The raw file mixes 4 different formats across rows on purpose.
# ---------------------------------------------------------------------
TS_FORMATS = [
    "%Y-%m-%d %H:%M:%S",
    "%d/%m/%Y %H:%M",
    "%m-%d-%Y %I:%M %p",
    "%d-%b-%Y %H:%M:%S",
]
```

The list of accepted timestamp formats is data, not control flow. Adding a fifth upstream system means appending one string rather than editing a branch - the difference between a change that is reviewable and one that is risky.

4.5 The loader

The loader executes the schema DDL and inserts the cleaned rows. It reads the schema from the `.sql` file rather than embedding DDL in Python, so the schema remains a first-class reviewable artefact.

Listing T.5 - schema execution and load scripts/build_database.py

```
"""
build_database.py
------------------
Creates the normalized SQLite database (database/complaints.db) from:
  - sql/schema.sql            (table definitions)
  - data/dim_category.csv
  - data/dim_channel.csv
  - data/dim_agent.csv
  - data/fact_complaints_clean.csv

Run this AFTER generate_data.py and clean_transform.py.
"""

import sqlite3
import pandas as pd
from pathlib import Path

ROOT = Path(__file__).resolve().parent.parent
DATA_DIR = ROOT / "data"
DB_PATH = ROOT / "database" / "complaints.db"
SCHEMA_PATH = ROOT / "sql" / "schema.sql"

DB_PATH.parent.mkdir(exist_ok=True)

# Fresh build every run
if DB_PATH.exists():
    DB_PATH.unlink()

conn = sqlite3.connect(DB_PATH)
cur = conn.cursor()

# 1. Create schema
with open(SCHEMA_PATH, "r", encoding="utf-8") as f:
    cur.executescript(f.read())
print("Schema created.")

# 2. Add an "Unassigned" placeholder agent for tickets with agent_id = -1
cur.execute("INSERT INTO dim_agent (agent_id, agent_name, team) VALUES (-1, 'Unassigned',
'Unassigned')")

# 3. Load dimension tables
dim_category = pd.read_csv(DATA_DIR / "dim_category.csv")
dim_channel = pd.read_csv(DATA_DIR / "dim_channel.csv")
dim_agent = pd.read_csv(DATA_DIR / "dim_agent.csv")

dim_category.to_sql("dim_category", conn, if_exists="append", index=False)
dim_channel.to_sql("dim_channel", conn, if_exists="append", index=False)
dim_agent.to_sql("dim_agent", conn, if_exists="append", index=False)
print(f"Loaded {len(dim_category)} categories, {len(dim_channel)} channels, {len(dim_agent)} agents.")

# 4. Load the fact table
fact = pd.read_csv(DATA_DIR / "fact_complaints_clean.csv")
fact.to_sql("fact_complaints", conn, if_exists="append", index=False)
print(f"Loaded {len(fact):,} complaint tickets into fact_complaints.")

conn.commit()
```

```
# 5. Sanity check: row counts + a quick preview query
check = pd.read_sql_query("""
    SELECT c.category_name, COUNT(*) AS tickets, ROUND(AVG(f.resolution_hours), 1) AS
avg_resolution_hrs,
           ROUND(100.0 * SUM(f.sla_breached) / COUNT(*), 1) AS sla_breach_pct
    FROM fact_complaints f
    JOIN dim_category c ON f.category_id = c.category_id
    GROUP BY c.category_name
    ORDER BY sla_breach_pct DESC
""", conn)
print("\nQuick sanity check - SLA breach % by category:\n")
print(check.to_string(index=False))

conn.close()
print(f"\nDatabase built at: {DB_PATH}")
```

WHY THE SCHEMA LIVES IN SQL AND NOT IN PYTHON

An ORM or an inline CREATE TABLE string hides the schema from anyone who does not read Python. A `.sql` file can be opened by a DBA, diffed meaningfully in a pull request, and run by hand against the database when something looks wrong. The schema is a contract, and contracts should be legible to everyone bound by them.

4.6 The verification stage

Between loading and reporting sits a stage most pipelines omit: running the analytical queries and checking the answers are sane before anything is published.

Listing T.6 - executing and sanity-checking the queries scripts/run_queries_check.py

```python
"""Quick verification runner: executes every query in sql/analysis_queries.sql
against the built database and prints results, so we can both (a) confirm the
SQL file has no syntax errors and (b) pull real numbers for the written
root-cause analysis report and README."""
import re
import sqlite3
import pandas as pd
from pathlib import Path

ROOT = Path(__file__).resolve().parent.parent
DB_PATH = ROOT / "database" / "complaints.db"
SQL_PATH = ROOT / "sql" / "analysis_queries.sql"

conn = sqlite3.connect(DB_PATH)
raw = SQL_PATH.read_text(encoding="utf-8")

blocks = [b.strip() for b in raw.split("\n\n") if b.strip()]

LABEL_RE = re.compile(r"^--\s*([A-E]\d+\.\s.+)$")

count = 0
for block in blocks:
    lines = block.split("\n")
    label = None
    sql_lines = []
    for line in lines:
        m = LABEL_RE.match(line.strip())
        if m:
            label = m.group(1)
        elif not line.strip().startswith("--"):
            sql_lines.append(line)
    sql = "\n".join(sql_lines).strip().rstrip(";")
    if not sql:
        continue
    try:
        df = pd.read_sql_query(sql, conn)
        count += 1
        print(f"\n=== {label or '(unlabeled query)'} ===")
        print(df.to_string(index=False))
    except Exception as e:
        print(f"\n!!! FAILED: {label}\n{sql}\nError: {e}")

print(f"\n\nTotal queries executed successfully: {count}")
conn.close()
```

A PIPELINE THAT CANNOT FAIL IS NOT A PIPELINE

If every stage always succeeds, the first indication of a problem is a wrong number in a meeting. A verification stage asserting invariants - compliance between 0 and 1, no negative durations, row counts within an expected band - converts a silent corruption into a loud build failure. Project 4 develops this into full quality gates and deliberately breaks them to prove they fire.

CHAPTER 5

Relational Modelling and the SQL Schema

> *Normalisation from first principles, why this project stops at third normal form, and the indexing decisions that make the analytical queries usable.*

5.1 Why a database at all, when pandas is already loaded

It is a fair question. The dataset fits comfortably in memory, and every query in this project could be a DataFrame operation. The database earns its place for reasons that are not about size:

- **The queries become an artefact.** A `.sql` file is readable by an analyst who does not know pandas, reviewable in a pull request, and reusable by a BI tool. A chain of DataFrame operations is none of those.
- **Constraints are enforced.** A schema with a primary key and typed columns rejects a duplicate ticket ID at write time. A DataFrame accepts it silently and the duplicate surfaces later as an inflated count.
- **It is the honest representation of the domain.** Complaints, agents, and teams are distinct entities with relationships. Flattening them into one wide table is a modelling choice that should be made deliberately, not by default.

5.2 Normalisation, explained properly

Normal forms are usually taught as rules to memorise. They are better understood as three successive answers to 'what could go wrong'.

First normal form (1NF): one value per cell

A cell containing `'fraud; refund'` cannot be filtered, grouped, or joined reliably. 1NF requires atomic values. The violation is easy to spot and easy to fix, and it is why the schema has one category per ticket rather than a delimited list.

Second normal form (2NF): no partial dependency on a composite key

If a table's key were (ticket_id, category) and the SLA target depended only on category, then the target would be duplicated across every ticket in that category. Update one and you have created an inconsistency. 2NF removes partial dependencies by splitting them out.

Third normal form (3NF): no transitive dependency

If a ticket record stored both `agent_id` and `agent_team`, then team depends on agent, which depends on ticket - a transitive chain. Moving an agent between teams would require updating every historical ticket, and any missed row silently corrupts every team-level aggregate.

Listing 4.1 - the full schema sql/schema.sql

```sql
-- schema.sql
-- Normalized SQLite schema for the UPI Payments Complaint & SLA Analytics
-- Dashboard project. A star-schema style design: one fact table
-- (fact_complaints) surrounded by small dimension tables, instead of one
-- giant flat spreadsheet. This mirrors how real companies structure data
-- warehouses so that BI tools and analysts can query it efficiently.

PRAGMA foreign_keys = ON;

DROP TABLE IF EXISTS fact_complaints;
DROP TABLE IF EXISTS dim_category;
DROP TABLE IF EXISTS dim_channel;
DROP TABLE IF EXISTS dim_agent;

CREATE TABLE dim_category (
    category_id       INTEGER PRIMARY KEY,
    category_name     TEXT NOT NULL,
    category_group    TEXT NOT NULL,
    sla_target_hours  INTEGER NOT NULL
);

CREATE TABLE dim_channel (
    channel_id    INTEGER PRIMARY KEY,
    channel_name  TEXT NOT NULL
);

CREATE TABLE dim_agent (
    agent_id    INTEGER PRIMARY KEY,
    agent_name  TEXT NOT NULL,
    team        TEXT NOT NULL
);

CREATE TABLE fact_complaints (
    ticket_id            INTEGER PRIMARY KEY,
    customer_id          TEXT NOT NULL,
    category_id          INTEGER NOT NULL,
    channel_id           INTEGER NOT NULL,
    agent_id             INTEGER,                -- nullable: some raw tickets arrive unassigned
    transaction_amount_inr REAL,
    opened_at            TEXT NOT NULL,           -- ISO 8601 'YYYY-MM-DD HH:MM:SS'
    closed_at            TEXT,                    -- ISO 8601 or NULL if still open
    status               TEXT NOT NULL CHECK (status IN ('Open','Closed')),
    resolution_hours     REAL,
    sla_target_hours     INTEGER NOT NULL,
    sla_breached         INTEGER NOT NULL CHECK (sla_breached IN (0,1)),
    root_cause           TEXT,
    is_incident_day      INTEGER NOT NULL CHECK (is_incident_day IN (0,1)),
    FOREIGN KEY (category_id) REFERENCES dim_category(category_id),
    FOREIGN KEY (channel_id)  REFERENCES dim_channel(channel_id),
    FOREIGN KEY (agent_id)    REFERENCES dim_agent(agent_id)
);

CREATE INDEX idx_complaints_opened_at   ON fact_complaints(opened_at);
CREATE INDEX idx_complaints_category    ON fact_complaints(category_id);
CREATE INDEX idx_complaints_channel     ON fact_complaints(channel_id);
CREATE INDEX idx_complaints_sla_breach  ON fact_complaints(sla_breached);
CREATE INDEX idx_complaints_status      ON fact_complaints(status);
```

5.3 Where this project deliberately stops normalising

Higher normal forms exist - BCNF, 4NF, 5NF - and this project does not pursue them. That is a decision, not an omission.

> **NORMALISATION OPTIMISES WRITES; ANALYTICS IS READ-HEAVY**
>
> Every additional normal form reduces update anomalies at the cost of more joins per query. That trade is correct for a transactional system taking thousands of writes a second. An analytical dataset is written once and read thousands of times, so past 3NF the trade inverts: you are paying join cost on every read to prevent update anomalies that a read-only dataset cannot have.
>
> This is precisely why analytical warehouses use denormalised star schemas. Projects 5, 13 and 15 in this portfolio do exactly that, and Chapter 13 draws the line between the two designs.

5.4 Indexing: what an index is and when it pays

An index is an auxiliary data structure - in SQLite, a B-tree - that maps column values to row locations. Without one, filtering requires reading every row: a full table scan, cost $O(n)$. With one, the engine descends the tree: cost $O(\log n)$.

$$O(n) \longrightarrow O(\log n)$$

For 15,700 rows that is roughly 15,700 comparisons against about 14. The index is not free, however - it consumes space and must be updated on every write - so it is created for columns that appear in `WHERE` and `JOIN` clauses, and not for columns that are only ever selected.

> **WHY THE GAIN IS SMALLER THAN IT LOOKS HERE**
>
> At 15,700 rows, SQLite will often scan faster than it can traverse an index, because the whole table fits in a few pages of memory and scanning is sequential. The indexes in this schema are justified by correctness of habit and by behaviour at scale, not by measured speedup on this dataset. Claiming a large performance win here would be an unmeasured claim, and this book does not make those.

CHAPTER 6

The SQL Analysis Layer

> *Eighteen queries in five sections, and the SQL constructs that make them possible: aggregation, conditional aggregation, CTEs, and window functions.*

6.1 How the query file is organised

The queries are grouped so that each section answers one class of operational question:

Table 5.1 The five query sections

Section	Focus	Example question
A	Overall KPIs	What is our compliance rate right now?
B	SLA breach analysis	Which category breaches most?
C	Volume and trend	Is the queue growing?
D	Channel and agent	Do some teams outperform?
E	Incident deep-dive	What happened on the worst day?

Grouping matters more than it appears. An analyst arriving at a 235-line SQL file needs to find the query relevant to a question in seconds; an unstructured file guarantees that someone rewrites a query that already exists, and the two versions then disagree.

6.2 Conditional aggregation: computing a rate in one pass

The central computation of the project - a compliance rate - is a conditional aggregate. The technique is worth understanding thoroughly because it replaces an entire class of clumsy alternatives.

Listing 5.1 - breach rate per category, single pass

```sql
SELECT
  complaint_category,
  COUNT(*)                                      AS tickets,
  SUM(CASE WHEN sla_breached = 1 THEN 1 ELSE 0 END) AS breaches,
  ROUND(100.0 * SUM(CASE WHEN sla_breached = 1 THEN 1 ELSE 0 END)
        / COUNT(*), 1)                          AS breach_rate_pct
FROM complaints
WHERE status = 'Closed'
GROUP BY complaint_category
ORDER BY breach_rate_pct DESC;
```

WHY NOT TWO QUERIES AND A DIVISION

The obvious alternative is one query counting breaches, another counting tickets, and division in Python. It gives the same answer and is worse in three ways: it reads the table twice, it can silently mismatch if the two `WHERE` clauses drift apart, and it moves business logic out of the auditable SQL artefact into a script. Conditional aggregation keeps the definition in one place.

THE 100.0 IS NOT DECORATION

In SQLite, as in most SQL dialects, integer divided by integer performs integer division: `3 / 4` is `0`. Multiplying by `100.0` first forces floating-point arithmetic. Omitting the decimal point produces a table of zeroes and hundreds - a bug that looks like a data problem and is not.

6.3 Common table expressions: naming a thought

A CTE is a named subquery declared with `WITH`. Its value is primarily human: it lets a complex query be read top to bottom as a sequence of named steps rather than deciphered from the inside out.

Listing 5.2 - finding anomalous days with a z-score, built from CTEs

```
WITH daily AS (
  SELECT DATE(created_at) AS day, COUNT(*) AS tickets
  FROM complaints GROUP BY 1
),
stats AS (
  SELECT AVG(tickets) AS mean_t,
         AVG(tickets * tickets) - AVG(tickets) * AVG(tickets) AS var_t
  FROM daily
)
SELECT d.day, d.tickets,
       ROUND((d.tickets - s.mean_t) / NULLIF(SQRT(s.var_t), 0), 2) AS z
FROM daily d CROSS JOIN stats s
WHERE (d.tickets - s.mean_t) / NULLIF(SQRT(s.var_t), 0) > 2
ORDER BY z DESC;
```

THE Z-SCORE, AND THE VARIANCE IDENTITY USED ABOVE

A z-score expresses a value as a number of standard deviations from the mean:

$$z = \frac{x - \mu}{\sigma}$$

The query computes variance using the identity $\mathrm{Var}(X) = \mathbb{E}[X^2] - (\mathbb{E}[X])^2$, which needs only two running sums and therefore one pass. Be aware that this form is numerically less stable than the two-pass formula when the mean is large relative to the spread - at the magnitudes here it is entirely safe, but it is the reason numerical libraries use Welford's algorithm instead.

NULLIF GUARDS A DIVISION THAT WILL OTHERWISE FAIL

`NULLIF(SQRT(s.var_t), 0)` converts a zero denominator to `NULL`, and division by `NULL` yields `NULL` rather than an error. Without it, a day-set with no variance aborts the query. Defensive division is a habit worth forming: the case that triggers it is always the edge case you did not test.

6.4 Window functions: comparing a row to its neighbourhood

Aggregation collapses rows. A window function computes across a set of rows while *keeping* each row - which is what trend analysis requires.

Listing 5.3 - moving average, day-on-day delta, and rank

```
SELECT
  day,
  tickets,
  AVG(tickets) OVER (ORDER BY day
                     ROWS BETWEEN 6 PRECEDING AND CURRENT ROW) AS ma7,
  tickets - LAG(tickets, 1) OVER (ORDER BY day)                AS delta,
  RANK()     OVER (ORDER BY tickets DESC)                      AS busiest_rank
FROM daily_volume
ORDER BY day;
```

Three constructs appear here and each solves a distinct problem:

- **Frame clause** (`ROWS BETWEEN 6 PRECEDING AND CURRENT ROW`) defines a trailing 7-day window. A 7-day period is chosen because complaint volume has a strong weekly cycle; a window that is not a whole number of weeks mixes weekday and weekend traffic and produces an oscillation that is an artefact of the window, not the data.
- **LAG** reaches into the previous row, giving day-on-day change without a self-join.
- **RANK** orders rows without collapsing them - note that `RANK` leaves gaps after ties while `DENSE_RANK` does not, which matters if the rank feeds anything downstream.

WHY A MOVING AVERAGE SMOOTHS

A trailing mean of window k is:

$$\mathrm{MA}_k(t) = \frac{1}{k}\sum_{i=0}^{k-1} x_{t-i}$$

If the underlying series is signal plus independent noise, averaging k terms divides the noise standard deviation by $\sqrt{k}$ while leaving a slowly-moving signal intact. The cost is lag: the average responds to a genuine step change only after roughly $k/2$ periods, which is why a smoothed series should never be the only view of an incident.

CHAPTER 7

The Eighteen Queries, Explained

> *Walking the analytical layer section by section: what each query answers, the construct it demonstrates, and the trap it avoids.*

7.1 Section A - overall KPIs

Three queries establish the headline position. A1 computes total tickets, share closed, and overall compliance in a single pass using conditional aggregation. A2 reports mean and median resolution time together - the pairing that exposes the long tail discussed in Chapter 2. A3 sums the transaction value represented by the complaint queue, converting an operational problem into a financial exposure a director can act on.

WHY A2 REPORTS TWO MEASURES OF CENTRE

Reporting the mean alone invites the reader to picture a typical case that does not exist. Reporting the median alone hides that a minority of cases are catastrophically slow. Reporting both, adjacent, forces the reader to notice the gap - and the gap is the finding.

7.2 Section B - SLA breach analysis

This section carries the root-cause work. Queries break the breach population down by category, by team, by category within team, and by assignment status. Every slice reports both a rate and a volume, so the reader can never see one without the other.

Listing Q.1 - rate and share side by side

```sql
SELECT
  assigned_team,
  COUNT(*)                                        AS tickets,
  SUM(sla_breached)                               AS breaches,
  ROUND(100.0 * SUM(sla_breached) / COUNT(*), 1)  AS breach_rate_pct,
  ROUND(100.0 * SUM(sla_breached)
        / (SELECT SUM(sla_breached) FROM complaints
           WHERE status = 'Closed'), 1)           AS share_of_breaches_pct
FROM complaints
WHERE status = 'Closed'
GROUP BY assigned_team
ORDER BY breach_rate_pct DESC;
```

THE SCALAR SUBQUERY MUST REPEAT THE OUTER FILTER

The subquery computing total breaches runs once and returns a single value used by every row. It has to repeat the outer WHERE clause, or the denominator covers a different population than the numerator and every share is quietly wrong. This is the most common way a percentage-of-total column becomes subtly incorrect, and it never raises an error.

7.3 Section C - volume and trend

Daily and monthly volume, moving averages, and the salary-window comparison. This is where the window functions of Chapter 5 do their work, and where the 36% payday spike was found.

Listing Q.2 - normalising by days, not comparing raw totals

```
WITH tagged AS (
  SELECT DATE(created_at) AS day,
         CASE WHEN CAST(STRFTIME('%d', created_at) AS INT) >= 25
                OR CAST(STRFTIME('%d', created_at) AS INT) <= 2
              THEN 'salary_window' ELSE 'rest_of_month' END AS period
  FROM complaints
)
SELECT period,
       COUNT(*)                                    AS tickets,
       COUNT(DISTINCT day)                         AS days,
       ROUND(1.0 * COUNT(*) / COUNT(DISTINCT day), 1) AS tickets_per_day
FROM tagged GROUP BY period;
```

WHY DIVIDING BY COUNT(DISTINCT DAY) IS ESSENTIAL

The salary window spans about 9 days a month and the rest spans about 21. Comparing raw ticket totals would show the rest of the month as busier and reverse the finding entirely. Any comparison between periods of unequal length must normalise by exposure - here, days. This is the same principle as PMPM in Project 15: the denominator is never optional.

7.4 Section D - channel and agent performance

Breaks performance down by intake channel and by agent. This is where the Simpson's paradox warning becomes operational: agent comparisons are reported within category, because agents do not receive a random mix of cases and an aggregate ranking would largely measure who was handed the hard work.

7.5 Section E - incident deep-dive

The final section isolates the single worst day and characterises it: what arrived, which categories spiked, whether resolution time degraded, and whether the queue recovered. This is the query set an operations lead runs the morning after a bad night, and it exists so the answer takes thirty seconds

rather than an afternoon.

CHAPTER 8

The Mathematics of SLA Compliance

> *A compliance rate is a sample proportion, and treating it as an exact number is the most common analytical error in this domain. This chapter puts error bars on it.*

8.1 Compliance as a proportion

Formally, with n closed tickets of which m met their window:

$$\hat{p} = \frac{m}{n}$$

This is an *estimate* of an underlying process rate, not a physical constant. Ask what would have happened if the same operation had run the same month again: a different set of tickets would have arrived, and $\hat{p}$ would have come out slightly differently. The size of that variation is what determines whether a movement between two periods means anything.

8.2 Confidence intervals, and why the textbook formula is wrong here

The interval usually taught is the Wald interval:

$$\hat{p} \pm z_{\alpha/2}\sqrt{\frac{\hat{p}(1-\hat{p})}{n}}$$

It is simple, it is what most people reach for, and it behaves badly exactly where operational analysts need it: small samples and proportions near 0 or 1. A category with 20 tickets and no breaches gets a Wald interval of [0, 0] - a claim of perfect certainty from twenty observations.

THE WILSON SCORE INTERVAL

Wilson inverts the hypothesis test instead of approximating the distribution, giving:

$$\frac{\hat{p} + \frac{z^2}{2n} \pm z\sqrt{\frac{\hat{p}(1-\hat{p})}{n} + \frac{z^2}{4n^2}}}{1 + \frac{z^2}{n}}$$

It never leaves [0, 1], it stays sensible at the extremes, and it is barely more work. For an operational dashboard reporting rates per category - where some categories have few tickets - it is simply the correct default.

The practical consequence for this project: a category-level breach rate computed from a few dozen tickets carries a wide interval, and ranking categories by point estimate alone will reorder them from month to month for no reason. Chapter 12 states which of the findings survive this scrutiny and which do not.

8.3 Rate versus volume: the mistake that misdirects budgets

The highest breach *rate* and the largest number of *breaches* are usually different categories, and confusing them sends resources to the wrong place.

Table 6.1 Rate and volume point at different interventions

Category	Breach rate	Share of all breaches	Reading
Wrong Beneficiary Credited	42.1% (highest)	small	worst experience per customer affected
Failed Transaction - Debited	lower	largest	worst total customer-hours lost

BOTH NUMBERS, ALWAYS, SIDE BY SIDE

A category with a 42% breach rate and 200 tickets produces 84 unhappy customers. One with an 18% rate and 4,000 tickets produces 720. The first is the more broken process; the second is the bigger problem. This project's root-cause report leads with the second for exactly this reason, and says so explicitly - which is why Root Cause #2 is the reversal process rather than the fraud queue, despite fraud having the uglier percentage.

8.4 Comparing two rates without fooling yourself

To ask whether Team A genuinely breaches more than Team B, compare the difference against its own standard error:

$$z = \frac{\hat{p}_A - \hat{p}_B}{\sqrt{\hat{p}(1-\hat{p})\left(\frac{1}{n_A} + \frac{1}{n_B}\right)}}$$

where $\hat{p}$ is the pooled rate across both groups. A difference of a few percentage points between two teams of a few hundred tickets each is frequently indistinguishable from noise, and presenting it as a performance gap invites an intervention that cannot work because there is nothing there to fix.

THE GAP THAT IS NOT NOISE

Fraud & Disputes at 40.9% against Technical Escalations at 6.0% is not a marginal difference; it is nearly a factor of seven, across thousands of tickets. No statistical test is needed to take that seriously. The discipline matters for the *small* differences, and the value of knowing the test is knowing when you do not need it.

8.5 Simpson's paradox: the aggregate that reverses

It is possible for Team A to breach less than Team B in every single category, and yet breach more overall. This is not a paradox in the mathematics; it is a paradox only to intuition.

The mechanism is composition. If Team A handles mostly fraud cases (hard, high breach rate) and Team B handles mostly reversals (easier), then A's aggregate is dominated by its hard mix even if it outperforms B on like-for-like work. Comparing the aggregates compares the case mix, not the teams.

> **THE DEFENCE**
>
> Never compare aggregate rates across groups that face different case mixes without either stratifying by category or standardising to a common mix. In this project the team comparison is reported alongside the per-category breakdown for precisely this reason, so a reader can see whether the ranking holds within categories as well as across them.

CHAPTER 9

Worked Numerical Examples

> *The statistics of Chapter 6 applied to this project's actual numbers, step by step, so the formulas become arithmetic you can check by hand.*

9.1 Example 1 - a confidence interval on the headline rate

Overall compliance is 75.7% over 15,713 tickets. How precise is that?

With $\hat{p} = 0.757$ and $n = 15713$:

$$SE = \sqrt{\frac{0.757 \times 0.243}{15713}} \approx 0.00342$$

The 95% interval is $0.757 \pm 1.96 \times 0.00342$, i.e. **[75.0%, 76.4%]**.

WHAT THIS TELLS AN OPERATIONS LEAD

The interval is about plus or minus 0.7 percentage points. A move from 75.7% to 76.2% next month is not evidence of improvement - it is inside the noise. A move to 78% would be. Knowing this in advance stops a team celebrating or panicking about a number that did not really change.

9.2 Example 2 - the same formula on a small slice

Now the 235 unassigned tickets at a 29.8% breach rate:

$$SE = \sqrt{\frac{0.298 \times 0.702}{235}} \approx 0.0299$$

The 95% interval is roughly **[23.9%, 35.7%]** - nearly twelve percentage points wide. The direction is safe (clearly worse than Technical Escalations at 6.0%); the precise figure is not.

WHY THE REPORT SAYS 'WORSE THAN ANY STAFFED TEAM' AND NOT '29.8%'

The defensible claim is the comparison, not the decimal. Writing the precise figure invites a reader to treat it as a measured constant, notice it is 26% next month, and conclude something changed.

9.3 Example 3 - comparing two teams properly

Tier-1 breaches 24.0% of 8,000 tickets; another team breaches 27.0% of 900. Is the difference real?

Pooled rate:

$$\hat{p} = \frac{0.240 \times 8000 + 0.270 \times 900}{8900} \approx 0.243$$

Standard error of the difference:

$$SE = \sqrt{0.243 \times 0.757\left(\frac{1}{8000} + \frac{1}{900}\right)} \approx 0.0152$$

So $z \approx 1.97$ - just past the conventional 1.96 threshold.

JUST PAST THE THRESHOLD IS NOT A STRONG FINDING

A z of 1.97 is a p-value near 0.049: technically significant and practically a coin-flip from not being. If this were one of twenty comparisons, a result this marginal is exactly what multiple testing produces by chance. Treat it as a hypothesis worth another month of data, not a finding worth reorganising a team over.

9.4 Example 4 - mean versus median on a lognormal

Suppose a category has $\mu = 2.5$ and $\sigma = 1.0$ in log-hours:

$$\text{median} = e^{2.5} \approx 12.2 \text{ hours}$$

$$\text{mean} = e^{2.5 + 0.5} \approx 20.1 \text{ hours}$$

The mean is 65% higher, and neither is wrong - they answer different questions. The median answers 'what is a typical case'; the mean answers 'total workload divided by case count'.

WHICH TO REPORT DEPENDS ON THE DECISION

For a customer-facing promise use a percentile - customers experience their own case, not an average. For capacity planning use the mean, because total agent-hours is a sum and the mean is the quantity that scales with volume. Reporting one and using it for the other decision is a genuine and common error.

CHAPTER 10

Root-Cause Analysis as a Method

> *Moving from 'the number is bad' to 'this specific mechanism is why, and here is what to change' - and the discipline that separates a cause from a correlation.*

10.1 Pareto: where the mass is

The Pareto principle - that a large share of effects come from a small share of causes - is a starting heuristic, not a law. Applied here it means ranking categories by *contribution to total breaches* and asking how few categories account for most of the problem.

The value is prioritisation under a real constraint: an ops team can change perhaps two processes a quarter. A ranked contribution list turns 'everything is a bit broken' into an ordered agenda.

10.2 Segmentation: slicing until the mechanism appears

A single aggregate hides everything. The method is to slice the breach population along each available dimension and look for a slice where the rate differs sharply from the base rate:

- by **category** - is it the type of problem?
- by **team** - is it who handles it?
- by **channel** - is it how it arrived?
- by **time** - is it when it arrived?
- by **assignment status** - was anyone actually looking at it?

The last of those is the one most analyses never try, and it produced this project's third root cause.

> **SLICING UNTIL SOMETHING APPEARS IS ALSO HOW FALSE FINDINGS ARE MADE**
>
> If you test enough slices, some will look significant by chance alone. Twenty independent slices at a 5% threshold will produce about one spurious 'finding' even if nothing is happening. The defences are to state the hypotheses before slicing where possible, to prefer slices with a plausible mechanism over ones that are merely extreme, and to report the number of comparisons made. A finding with a story attached - unassigned tickets have no owner watching the clock - is far more trustworthy than one that is only a large number.

10.3 Correlation, confounding, and the limits of this design

This project is observational. It can establish that the Fraud & Disputes queue breaches more; it cannot establish that moving a ticket to that queue *causes* a breach, because the tickets are not randomly assigned - harder cases go there by design.

$$P(\text{breach} \mid \text{team}) \neq P(\text{breach} \mid do(\text{team}))$$

The notation on the right is Pearl's *do*-operator: the probability of breach if we intervened to set the team, as opposed to merely observing it. Observational data gives the left; decisions require the right.

HOW THIS PORTFOLIO ADDRESSES IT LATER

Establishing the right-hand quantity needs either randomisation or a credible identification strategy. Project 10 uses propensity score matching and difference-in-differences on a delivery programme; Project 13 uses difference-in-differences on a maintenance rollout and shows the naive comparison erasing the entire true effect. This project stops at description and says so - which is the honest position, not a weakness to be papered over.

10.4 From finding to recommendation

A finding becomes a recommendation only when it names a mechanism and a change. The three in this project:

Table 7.1 Findings mapped to mechanisms and changes

Finding	Mechanism	Recommended change
Fraud queue breaches at 40.9%	one SLA applied to cases of very different complexity	tiered triage: fast-track simple disputes, longer explicit SLA for complex
Reversals are the largest breach volume	manual handling of a mechanical process	auto-reversal for gateway timeouts; escalate at T+18h
Unassigned tickets breach at 29.8%	no owner means no one watching the clock	auto-escalate anything unassigned beyond 30 minutes

WHY EACH RECOMMENDATION NAMES A TRIGGER

'Improve fraud handling' is not actionable. 'Any transaction still unreversed at T+18 hours is proactively flagged' is: it names a condition a system can evaluate, a time, and an action. The test of a recommendation is whether an engineer could implement it without asking a follow-up question.

CHAPTER 11

Running the Project, Step by Step

> *Every terminal command needed to go from a fresh clone to a running dashboard, with what each one does and what correct output looks like.*

11.1 Prerequisites

You need Python 3.11 or newer and Git. Nothing else - no database server, no cloud account, no paid service. Verify:

Check your toolchain

```
python --version
git --version
```

On Windows, if `python` is not recognised, use the full interpreter path or install from python.org with 'Add to PATH' ticked.

11.2 Step 1 - Clone and enter the repository

Get the code

```
git clone https://github.com/Nikhil201716/01-UPI-Payments-Complaint-SLA-Dashboard.git
cd 01-UPI-Payments-Complaint-SLA-Dashboard
```

11.3 Step 2 - Create an isolated environment

A virtual environment keeps this project's dependencies from colliding with anything else on your machine.

Create and activate a virtual environment

```
python -m venv .venv

# Windows (PowerShell)
.venv\Scripts\Activate.ps1

# macOS / Linux
source .venv/bin/activate
```

IF POWERSHELL REFUSES TO RUN THE ACTIVATION SCRIPT

Windows blocks unsigned scripts by default. Allow them for your user account only:

```
Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned
```

11.4 Step 3 - Install dependencies

Install

```
pip install -r requirements.txt
```

This pulls pandas, numpy, openpyxl, streamlit and matplotlib. Expect it to take a minute or two on a first run.

11.5 Step 4 - Run the whole pipeline

One command runs every stage in dependency order - generate, clean, load, verify, build dashboards:

The full build

```
python scripts/run_pipeline.py
```

What it does, in order:

1. **generate_data.py** - writes the raw synthetic ticket extract
2. **clean_transform.py** - parses timestamps, handles missingness, computes resolution time and the breach flag
3. **build_database.py** - creates the SQLite schema and loads the cleaned data
4. **run_queries_check.py** - executes the analytical queries and verifies they return sane results
5. **build_excel_dashboard.py** - writes the formatted XLSX

EXPECTED OUTPUT

Each stage prints a line confirming rows written or checks passed. The run finishes with the headline figures - roughly 15,700 tickets loaded and an overall SLA compliance rate near 75.7%. If your numbers differ, the seed has changed or a stage failed silently; re-run the stages individually to find where.

11.6 Step 5 - Run the stages individually (optional)

Useful when developing or debugging a single stage:

Each stage on its own

```
python scripts/generate_data.py
python scripts/clean_transform.py
python scripts/build_database.py
python scripts/run_queries_check.py
python dashboard/build_excel_dashboard.py
```

11.7 Step 6 - Query the database directly

The whole point of the database is that you can interrogate it yourself. If you have the SQLite CLI:

Explore the database interactively

```
sqlite3 database/complaints.db

-- inside the sqlite prompt:
.tables
.schema complaints
SELECT COUNT(*) FROM complaints;
.quit
```

No SQLite CLI installed? Python has it built in:

Query without installing anything

```
python -c "import sqlite3, pandas as pd; "
  "print(pd.read_sql('SELECT complaint_category, COUNT(*) n "
  "FROM complaints GROUP BY 1 ORDER BY n DESC', "
  "sqlite3.connect('database/complaints.db')))"
```

11.8 Step 7 - Launch the interactive dashboard

Start the Streamlit app

```
streamlit run dashboard/streamlit_app.py
```

Streamlit prints a local URL - normally `http://localhost:8501` - and opens your browser. Stop it with `Ctrl+C` in the terminal.

IF PORT 8501 IS ALREADY IN USE

```
streamlit run dashboard/streamlit_app.py --server.port 8springs502
```

11.9 Step 8 - Open the Excel dashboard

The generated workbook is at `reports/UPI_Complaint_SLA_Dashboard.xlsx`. Open it with Excel, LibreOffice Calc, or Google Sheets. It needs no running process and can be emailed as-is.

11.10 Command reference card

Every command in this chapter, in one place, for when you already know what you want and just need the incantation.

Complete command reference

```
# --- setup (once) -------------------------------------------
git clone https://github.com/Nikhil201716/01-UPI-Payments-Complaint-SLA-Dashboard.git
cd 01-UPI-Payments-Complaint-SLA-Dashboard
python -m venv .venv
.venv\Scripts\Activate.ps1          # Windows PowerShell
source .venv/bin/activate            # macOS / Linux
pip install -r requirements.txt

# --- build everything ---------------------------------------
python scripts/run_pipeline.py

# --- or run stages individually -----------------------------
python scripts/generate_data.py           # raw synthetic extract
python scripts/clean_transform.py         # parse, clean, derive
python scripts/build_database.py          # schema + load
python scripts/run_queries_check.py       # verify the answers
python dashboard/build_excel_dashboard.py # formatted XLSX

# --- explore ------------------------------------------------
sqlite3 database/complaints.db            # interactive SQL
streamlit run dashboard/streamlit_app.py  # interactive dashboard

# --- deactivate when finished -------------------------------
deactivate
```

THE ONE COMMAND THAT MATTERS

If you remember nothing else: `python scripts/run_pipeline.py` from an activated environment rebuilds the entire project from nothing and prints the headline figures. Everything else in this chapter is either setup that happens once or a way to inspect what that command produced.

11.11 Troubleshooting

Table 8.1 Common problems and their fixes

Symptom	Cause	Fix
`ModuleNotFoundError`	environment not activated	re-run the activate command from Step 2
`no such table: complaints`	database stage not run	run <code>python scripts/build_database.py</code>
Excel file missing	dashboard stage failed	run the dashboard script alone and read the traceback
Numbers differ from the book	regenerated with a changed seed	check <code>SEED</code> in <code>generate_data.py</code> is 42
`streamlit: command not found`	not installed in this env	`pip install streamlit` with the env active

CHAPTER 12

The Excel Dashboard

> *Building a genuinely useful spreadsheet with openpyxl: layout, conditional formatting, and why the artefact that needs no software running is often the one that gets used.*

12.1 openpyxl and the object model

openpyxl represents a workbook as a tree: a `Workbook` holds `Worksheet` objects, which hold `Cell` objects, each carrying a value and a style. Writing a dashboard means walking that tree deliberately rather than dumping a DataFrame.

Listing 9.1 - styling a header row — dashboard/build_excel_dashboard.py

```
def style_header_row(ws, row_idx, n_cols):
```

Listing 9.2 - writing a DataFrame cell by cell — dashboard/build_excel_dashboard.py

```
def write_df(ws, df, start_row=1, start_col=1):
```

12.2 Conditional formatting: encoding the threshold in the file

A compliance figure below target should be visible without reading it. Conditional formatting rules live inside the workbook, so the colouring survives being emailed, re-sorted, and filtered - which a colour written into static cell styles does not.

> **RULES TRAVEL; STATIC COLOURS DO NOT**
>
> If you colour a cell red because its value is 68%, and someone later sorts or filters the sheet, the red stays with the cell while the meaning moves. A rule reevaluates against whatever value the cell holds, so the sheet stays truthful under manipulation by someone who was not told the rules.

12.3 Layout decisions that make a sheet usable

- **Freeze panes** below the header so column names stay visible while scrolling - the single highest-value line of code in a wide sheet.
- **Column widths** set from content length; a column showing `####` is a bug report waiting to happen.
- **Number formats** applied explicitly - percentages as percentages, currency with separators. A raw `0.757` where a reader expects `75.7%` is a misread waiting to happen.

- **One sheet per question**, not one giant grid. Tabs are the cheapest navigation affordance in existence.

CHAPTER 13

The Streamlit Dashboard

> *Streamlit's re-execution model, why it surprises people coming from callback frameworks, and the caching that makes it viable.*

13.1 The execution model

Streamlit re-runs the entire script top to bottom on every interaction. There are no callbacks and no component tree to manage: a widget returns its current value, and the code below it runs again with that value.

Listing 10.1 - app setup and data loading dashboard/streamlit_app.py

```
"""
streamlit_app.py
----------------
Interactive web dashboard for the UPI Payments Complaint & SLA Analytics
project. Reads directly from the SQLite database and lets a support-ops
stakeholder filter by date range, category, and channel to instantly see
SLA compliance, resolution-time trends, and root-cause breakdowns.

Run with:
    streamlit run dashboard/streamlit_app.py
"""

import sqlite3
from pathlib import Path

import pandas as pd
import plotly.express as px
import streamlit as st

ROOT = Path(__file__).resolve().parent.parent
DB_PATH = ROOT / "database" / "complaints.db"

st.set_page_config(page_title="UPI Complaint & SLA Dashboard", layout="wide", page_icon="📊")


@st.cache_data
def load_data():
    conn = sqlite3.connect(DB_PATH)
    df = pd.read_sql_query("""
        SELECT f.ticket_id, f.customer_id, c.category_name, c.category_group,
               ch.channel_name, a.agent_name, a.team, f.transaction_amount_inr,
               f.opened_at, f.closed_at, f.status, f.resolution_hours,
               f.sla_target_hours, f.sla_breached, f.root_cause, f.is_incident_day
        FROM fact_complaints f
        JOIN dim_category c ON f.category_id = c.category_id
        JOIN dim_channel ch ON f.channel_id = ch.channel_id
        JOIN dim_agent a ON f.agent_id = a.agent_id
    """, conn)
    conn.close()
    df["opened_at"] = pd.to_datetime(df["opened_at"])
```

WHY THIS MODEL IS A GOOD TRADE FOR ANALYTICS

It eliminates an entire category of bug - state that disagrees with what is displayed - because the display is always recomputed from current state. The cost is that everything re-runs, including expensive work, which is what caching exists to solve.

13.2 Caching, and the correctness question underneath it

Decorating a loader with `@st.cache_data` stores the result keyed by the function's arguments, so a filter change does not re-read the database.

CACHE KEYS MUST INCLUDE EVERYTHING THE RESULT DEPENDS ON

The cache key is built from the arguments. If a function reads a file path from a global instead of taking it as an argument, changing that global will not invalidate the cache, and the app will serve stale data with total confidence. Pass every dependency as an argument - this is the same discipline as pure functions, enforced by a performance feature.

13.3 Streamlit and Excel are complements, not alternatives

Table 10.1 Each format is right for a different moment

	Streamlit	Excel
Needs a running process	yes	no
Works offline / emailable	no	yes
Interactive filtering	yes	limited
Auditable by a regulator	hard	easy
Good for exploration	yes	no
Good for a Sunday incident review	no	yes

CHAPTER 14

Results

> *('The numbers the pipeline actually produced, what they support, and - equally important - what they do not.',)*

14.1 Headline figures

Table 11.1 Headline results

Measure	Value	Comment
Tickets analysed	**15,713**	12-month synthetic window
Overall SLA compliance	**75.7%**	against a 90% target
Implied breach rate	**~1 in 4**	the operational headline
Worst team	**Fraud & Disputes, 40.9%**	vs 6.0% Technical Escalations
Unassigned tickets	**235 (1.5%)**	29.8% breach rate
Salary-window volume	**53.5/day**	vs 39.3/day otherwise, +36%

14.2 The three root causes

Root Cause 1 - the Fraud & Disputes queue is structurally the weakest link. A 40.9% breach rate, consistent across both categories that team owns: Wrong Beneficiary Credited at 42.1% (120.7 hours against a 120-hour target) and Unauthorized Transaction at 40.8% (175.8 against 168). The consistency across two categories is what makes this structural rather than a bad month.

Root Cause 2 - failed-transaction reversals are the largest source of raw breach volume. Not the worst rate - the worst total. This is the rate-versus-volume distinction from Chapter 6 doing real work: the biggest improvement available is in a category whose percentage looks unremarkable.

Root Cause 3 - unassigned tickets breach more than any staffed team. 235 tickets, 29.8% breach rate, worse than Tier-1's 24.0%. A ticket with no owner has nobody watching its clock.

14.3 What these results do not establish

READ THIS BEFORE QUOTING ANY NUMBER ABOVE

The data is synthetic. These figures describe a generated dataset built to behave like a payments queue. They are not measurements of any real company, and the specific percentages should never be cited as industry benchmarks.

The design is observational. Team differences reflect both capability and case mix, and this project cannot separate them. The recommendation to tier fraud triage is reasonable, but its effect size is unknown until it is tested.

Some slices are small. The 235 unassigned tickets are enough to notice and not enough to characterise precisely; the Wilson interval on 29.8% from 235 observations is roughly ±6 percentage points. The direction is solid, the exact figure is not.

Stating limitations plainly is not hedging. An analysis that claims more than its design supports will eventually be checked by someone, and everything else in it becomes suspect at that moment.

CHAPTER 15

Exercises

> *Questions to test whether the material landed, with worked answers. Attempt them against the actual database before reading the solutions.*

15.1 Set A - SQL

1. Write a query returning, per complaint category, the ticket count, the breach rate, and that category's share of all breaches, ordered by share descending.
2. Modify it to cover only tickets created in the final 90 days of the data.
3. Find every day whose ticket count exceeds the trailing 7-day average by more than 50%.
4. Find the best-performing agent within each category, and explain why the within-category restriction matters.

Answers

A1. Listing Q.1 with `complaint_category` substituted for `assigned_team`. The scalar subquery computing total breaches must repeat the `status = 'Closed'` filter.

A2. Add a predicate deriving the window from the data's own maximum date rather than hard-coding one, so the query stays correct when the data is regenerated.

A3. A CTE producing daily counts, then a window function for the trailing average, then an outer filter comparing them. The comparison cannot sit in the same SELECT that defines the window function - window functions are evaluated after WHERE, so it needs a wrapping query or a second CTE.

A4. Group by category and agent, then `RANK() OVER (PARTITION BY complaint_category ORDER BY breach_rate)`. The restriction matters because agents do not receive a random case mix; an overall ranking would substantially measure who was assigned easier work.

15.2 Set B - statistics

1. A category shows a 0% breach rate over 12 closed tickets. What is the Wald interval, and why is it obviously wrong?
2. Compliance moves from 75.7% to 76.9% month on month over roughly 1,300 tickets a month. Is that a real improvement?
3. A team's mean resolution time is 48 hours and its median is 9. What is happening, and what should be reported to leadership?

Answers

B1. Wald gives [0, 0] - a claim of certainty from twelve observations. Wilson gives roughly [0%, 24%], honestly reflecting that twelve clean cases tell you very little. This is the concrete reason Chapter 6 makes Wilson the default.

B2. At about 1,300 tickets a month the standard error on each month is roughly 1.2 percentage points, so the SE of the difference is about 1.7. A 1.2-point move is well inside that. Not a real improvement - and reporting it as one burns credibility when it reverses.

B3. A mean five times the median indicates a heavy right tail: most cases close in about 9 hours, a minority take enormously longer. Report the median as the typical experience, a high percentile as the worst-case promise, and investigate the tail as a separate population - it is almost certainly a different kind of problem, not slow versions of the same one.

15.3 Set C - design and judgement

1. The pipeline excludes open tickets from the compliance calculation. Argue both sides, then state which you would ship.
2. A stakeholder wants one number for a wall. Which do you give, and what must accompany it?
3. The Fraud team breaches at 40.9%. Give two explanations that do not involve the team performing poorly.

Answers

C1. *For inclusion:* a ticket open past its window has already breached, and excluding it flatters the number - a long-running case can be hidden indefinitely by never closing it. *For exclusion:* a ticket open two hours against a 168-hour window has not breached, and counting it as either outcome is wrong. *Ship:* exclude open tickets from the rate, and separately report open tickets already past their window as a prominent distinct figure - capturing the real risk without corrupting the ratio.

C2. Overall compliance, with the target it is measured against, the population it covers, and its confidence interval. A bare percentage on a wall becomes a target to be managed rather than a measurement, and the easiest way to improve it is to change the population quietly.

C3. First, *case mix*: fraud investigation genuinely takes longer and depends on external parties, so a 168-hour window may simply be too tight for the work. Second, *routing*: if the hardest cases from other categories are escalated into that queue, its population is adversely selected by construction. Both are structural, and both imply a different intervention than a performance conversation - which is exactly why the recommendation is tiered triage plus a realistic explicit SLA, not 'work faster'.

CHAPTER 16

Appendix

> *('Repository map, glossary, and where to go next.',)*

16.1 A - Repository map

Table 12.1 Where everything lives

Path	Purpose
scripts/generate_data.py	synthetic ticket generator, seeded
scripts/clean_transform.py	timestamp parsing, missingness, derived fields
scripts/build_database.py	schema creation and load
scripts/run_queries_check.py	executes and sanity-checks the queries
scripts/run_pipeline.py	orchestrates every stage in order
sql/schema.sql	table definitions, keys, indexes
sql/analysis_queries.sql	18 analytical queries in 5 sections
dashboard/build_excel_dashboard.py	formatted XLSX writer
dashboard/streamlit_app.py	interactive dashboard
reports/root_cause_analysis.md	the written analysis

16.2 B - Glossary

Table 12.2 Terms used in this book

Term	Meaning
UPI	Unified Payments Interface, India's real-time payments rail
NPCI	National Payments Corporation of India, operator of UPI
SLA	Service Level Agreement - a resolution-time promise
Breach	a ticket resolved after its category's target window
Compliance rate	share of closed tickets resolved within target
CTE	Common Table Expression - a named subquery via WITH
Window function	computes across rows without collapsing them
Lognormal	distribution of a variable whose log is normal
Wilson interval	a confidence interval for a proportion that behaves well near 0 and 1
Simpson's paradox	an aggregate trend that reverses within every subgroup
Pareto analysis	ranking causes by their contribution to an effect
do-operator	Pearl's notation for intervention, distinct from observation

16.3 C - SQL construct quick reference

Every construct used in this project, with the thing that most often goes wrong with it.

Table 12.3 SQL constructs used in this project

Construct	What it does	The trap
`GROUP BY`	collapses rows into groups	every non-aggregated column in SELECT must appear here, or the engine picks an arbitrary row
`CASE WHEN` inside `SUM`	counts rows meeting a condition	the ELSE branch must be 0, not NULL, or the count silently shrinks
`100.0 *` before division	forces float arithmetic	integer / integer truncates to 0 in SQLite and most dialects
`WITH` (CTE)	names a subquery	a CTE referenced twice may be evaluated twice; materialise if expensive
`OVER (ORDER BY ...)`	window function	evaluated after WHERE, so it cannot be filtered in the same SELECT
`ROWS BETWEEN n PRECEDING`	frame for a moving window	ROWS counts rows; RANGE counts values - they differ whenever there are gaps or ties
`LAG` / `LEAD`	reach to an adjacent row	returns NULL at the boundary; wrap in COALESCE if it feeds arithmetic
`RANK` vs `DENSE_RANK`	ordinal position	RANK leaves gaps after ties, DENSE_RANK does not
`NULLIF(x, 0)`	guards a division	returns NULL not an error - which then propagates silently
`COUNT(*)` vs `COUNT(col)`	row count vs non-null count	they differ exactly when the column has nulls, which is when it matters
`LEFT JOIN`	keeps unmatched left rows	a WHERE clause on the right table silently converts it to an INNER JOIN
scalar subquery in SELECT	one value reused per row	must repeat the outer WHERE or numerator and denominator disagree
`STRFTIME`	extract a date part in SQLite	returns text; CAST to INT before numeric comparison
`DISTINCT`	de-duplicates the whole row	often masks a join fan-out rather than fixing it

THE LAST ROW IS THE ONE WORTH REMEMBERING

Reaching for `DISTINCT` because a count looks too high is the single most common way a join defect gets buried instead of fixed. The duplicate rows are evidence; removing them without asking where they came from means the same fan-out will corrupt the next aggregate that does not happen to be de-duplicated. Projects 13 and 15 add explicit fan-out assertions to the warehouse build for exactly this reason.

16.4 D - Where this leads next

This project is the analyst foundation of a fifteen-project portfolio. The techniques introduced here recur, and deepen:

- **Project 2** takes segmentation further with RFM and a churn model.
- **Project 3** takes the time-series thread into forecasting.
- **Project 4** turns this pipeline into an orchestrated Airflow DAG with quality gates.
- **Project 5** replaces the hand-built schema with dbt and a warehouse.
- **Project 13 and 15** return to the metric-definition problem raised in Chapter 6 and solve it properly with a governed semantic layer.

The thread running through all of them is the discipline this book has tried to demonstrate: state what you are measuring, measure it against something known, and report the limitations as prominently as the findings.